

Milestone 2 Multi-Agent Workflow design document

For this milestone, I implemented a multi-agent workflow into my AI Hiring Assistant system. The goal of this system is to help hiring managers evaluate job candidates by processing job requirements and a candidate's application information. This system uses multiple agents, where each agent has its own role and is responsible for a specific task.

Agent Architecture:

This system follows a fixed pipeline structure, which consists of:

User Input -> Form Analysis Agent -> Evaluation Agent -> Summarization Agent -> Output

Each agent takes the output of the previous agent and sends the results forward to the next agent.

Agent Roles:

Form Analysis Agent:

This agent's role is to extract basic information from the candidate application, such as name, location, and job skills, converting raw candidate text into structured output data.

Evaluation Agent:

This agent's role is to compare the candidate's qualifications with the job requirements. This agent uses the Gemini API to evaluate a candidate and generate their strengths, weaknesses, and a recommendation for the candidate, therefore converting structured data from the Form Analysis Agent and the Job requirements posted by the user into evaluated results for a candidate's application.

Summarization Agent:

This agent formats the evaluation into a clean output for the user, taking the input from the evaluation results and summarizing them into a final candidate evaluation.

The system also uses a sequential pipeline, where the output of one agent becomes the input for the next agent. Each agent runs in order and the process comes to an end after the summarization step.

Tool/API integration:

This system uses the Gemini API, specifically used by the evaluation agent. This allows the agent to use an external AI tool for reasoning and generating candidate evaluations.

Handling Failures:

The system handles basic issues such as:

- Missing input fields by asking the user to fill out the missing input fields
- Incomplete candidate data by treating it as a weakness.

Implementation:

The system was implemented using Python and Streamlit. Each agent is implemented as a function, and the pipeline runs when the user clicks the “Analyze Candidate” button.

Risks	Mitigation Plan	
	Design & development of an agent	Agent integration into processes
Adversarial attacks	Check inputs and keep prompts organized so the system only analyzes the provided data.	Limit access to trusted users and monitor suspicious activity
Bias and discrimination	Focus only on skills, experience, and qualifications, avoid any complex attributes, and stick to the task.	Keep a human in the loop to review outputs and fairness
Misinformation and hallucinations	Prompt the agent to only use the information that is provided, no outside information, and no hallucinations	Allow users to review outputs and make changes if needed.
Data privacy violations	Do not store any sensitive data, and only process information that is required	Restrict access and do not share candidate data publicly.